

Doc. no. P0842R0
Date: 2017-10-16
Project: Programming Language C++
Audience: Core Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Unknown Exports by Example

Table of Contents

- [Revision History](#)
 - [Revision 0](#)
- [1 Introduction](#)
- [2 Stating the problem](#)
- [3 Propose Solution](#)
- [4 Formal Wording](#)
- [5 References](#)

Revision History

Revision 0

Original version of the paper for the 2017 pre-Albuquerque mailing.

1 Introduction

The modules TS adds the ability to export user declarations that are visible in other translation units via an import declaration. It is not clear what the intended semantics are when exporting declarations that rely on names that are not also exported. Examples would clarify the intent.

This paper does **not** seek to open a design discussion on the topic it discusses. That would be appropriate material for follow-up papers targeting evolution after the TS is published. It seeks only clarity on the model that is proposed in the existing draft TS.

2 Stating the problem

The problem is most easily explored with an incremental set of examples. The classic example would be a box class that comprises two points.

```
module Shapes;

struct Point {
    // This class has module linkage
    int x;
    int y;
}

export class Box {
    // Private details depend on Point
    Point d_topLeft;
```

```

    Point d_extent;

public:
    // Public interface does not expose Point
    Box();

    int top() const;
    int bottom() const;
    int left() const;
    int right() const;
};

```

In practice, this example is just a little too simple, as the above code is a diagnosable error according to 10.7.1 [dcl.module.interface] p2

Every name introduced by an *export-declaration* shall have external linkage. If that declaration introduces an entity with a non-dependent type, then that type shall have external linkage or shall involve only types with external linkage.

In the exported class `Box`, the member `d_topLeft` is a declaration that introduces an entity (the member) of non-dependant type (as no templates are involved, there can be no type-dependency). As the class `Point` has module linkage, rather than internal linkage, the program (or module in this case) is ill-formed, and a diagnostic is required. It would be helpful to add this case to the example following this text in p2.

However, with a tiny tweak, this example becomes well-formed:

```

struct Point {
    // This class has external linkage and is owned by the global module
    int x;
    int y;
}

module Shapes;

export class Box {
    // Private details depend on Point
    Point d_topLeft;
    Point d_extent;

public:
    // Public interface does not expose Point
    Box();

    int top() const;
    int bottom() const;
    int left() const;
    int right() const;
};

```

This example is similar to what is expected to be the common case of class `Point` being supplied through a third party header for a pre-modules library. This demonstrates a module that exports a class that has no dependency on user defined types in its public interface, but requires an non-exported type for its implementation. Questions arise on how this might be used. The first question is whether is is legal export a class declared in such a manner? As far as I can tell, nothing in the TS would outlaw it, so I would expect it to compile. A similar effect can be achieved by importing a dependency on another module, but not exporting that dependency:

```

// First module:
module Primitives;

export struct Point {
    // This class is exported with external linkage

```

```

    int x;
    int y;
}

// Second module
import Primitives;

module Shapes;

export class Box {
    // Private details depend on Point, which is not exported from Shapes
    Point d_topLeft;
    Point d_extent;

public:
    // Public interface does not expose Point
    Box();

    int top() const;
    int bottom() const;
    int left() const;
    int right() const;
};

```

One notional implementation might emit size and alignment requirements for class `Box` without the detailed description of its private data. Of course, we might also want to retain information regarding whether it is trivial, standard layout, and other language-specific properties that matter to type traits. Assuming this works, we can try to import this class into another translation unit.

```

import module Shapes;
import module std.io;

int main() {
    Box x;
    std::cout << x.top() << ", " << x.left() << std::endl;
}

```

Is this client application expected to compile and run? If the class `Box` were parsed from a header file, we would expect the compiler to complain about the missing definition of `Point`. However, the compiler has already parsed that class definition and emitted a module interface file. Nothing in the client code makes direct use of the unknown `Point` class, so we might expect this to parse, compile, and run. As a non-Core language expert, it is not clear whether this simple example is expected to compile or not, and an example in the document would greatly improve clarity.

If we presume the above example does compile and run, we can ask some more interesting questions of this code:

```

import module Shapes;
import module std.io;

int main() {
    Box x;
    auto y = x.d_topLeft; // error due to access control
}

```

This code is clearly an error, because access control will not grant permission to directly use the private data members, and we cannot inject friendship from outside the system. However, there is also the question of whether `auto y` deduces a type before failing on access control, or whether the declaration should fail for other reasons, trying to deduce an unknown or incomplete type. This question becomes more relevant as introspection and reflection techniques are added to the language. However, using just C++11 we can make the question more

interesting by adding an overloaded function to Box whose presence affects name-lookup, even though one overload is private:

```
struct Point {  
    // This class has external linkage and is owned by the global module  
    int x;  
    int y;  
  
    Point(const char *); // converting constructor parses x,y from a string  
}  
  
module Shapes;  
  
export class Box {  
    // Private details depend on Point  
    Point d_topLeft;  
    Point d_extent;  
  
    void call(Point p, int param);  
  
public:  
    // Public interface does not expose Point, but overloads 'call'  
    Box();  
  
    void call(std::string p, double param);  
  
    int top() const;  
    int bottom() const;  
    int left() const;  
    int right() const;  
};
```

And the client application will now try to call on a Box:

```
import module Shapes;  
import module std.io;  
  
int main() {  
    Box x;  
    x.call("Hello", 3);  
}
```

Should this code compile? Without the exported definition of Point, we don't know if the private function is viable. If it is a viable candidate, it will also be the strongest match, and the program should not compile. However, if the conversion from 'const char *' to Point is not viable, then the public function is the strongest match, and the code is expected to compile and run.

My own mental model so far has been to treat Point as an incomplete type, as if it had been forward declared. This allows a certain subset of operations to work, and others to fail predicatably. I believe it puts the last example into a third category which is that if code in a (larger) complete program gets to see an overload set that resolves differently if all types are complete, and different translation units call that overload set in contexts that vary whether or not the type is complete, then we produce an ODR violation - an ill-formed program, but no diagnostic is required. However, this mental model does not serve well for exporting classes with data members of an incomplete type, as the point of parse failure would appear to be the import declaration - which should be able to rely on the interface file being emitted from a complete parse, and so have no unresolved names.

3 Propose Solution

The proposed solution is to add a small example or two to the TS that clarifies the case of exporting declarations that depend on non-exported entities. Wording will follow once the semantics are clear, and the minimal useful set of examples has been identified.

4 Formal Wording

Wording will follow in a revise paper, see above.

5 References

- [N4511](#) Latest C++ Working Paper at time of this paper
- [N4489](#) Modules TS Ballot Document